PRISM: PRior-guided Imagination Sampling in world Models

Anonymous authors

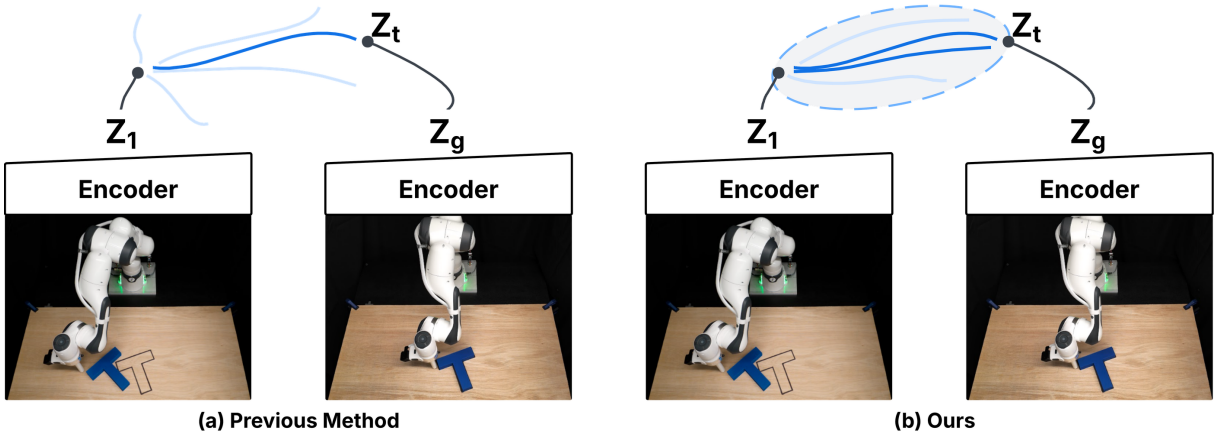


Fig. 1. **Planning with physical and action intuition.** Left: A planner samples candidate action sequences from an uninformed distribution and uses the world model's physical intuition to predict their outcomes. Right: PRISM also extracts action intuition from the same representations to guide sampling.

***Abstract*— Latent world models learn representations that enable them to predict how actions change the environment—a capability we call physical intuition. We find that these same representations also support action intuition: a lightweight head can learn a goal-conditioned action prior directly from the frozen encoder's features. We introduce PRISM to investigate how action intuition can be extracted from frozen world-model representations and used to guide goal-conditioned planning. The prior's mean and predicted uncertainty guide the search for useful actions, while the world model evaluates their consequences against the goal. Experiments on four simulated control tasks and two real-robot platforms show that a lightweight action head improves planning performance, and that the same fusion interface accepts a diffusion prior over the same frozen features without modification. These results demonstrate the potential of reusing world-model representations to both predict action consequences and guide action selection.**

***Index Terms*— World Models, Joint-Embedding Predictive Architecture (JEPA), Sampling-Based Planning, Action Prior**

I. Introduction

Predicting how actions change the environment helps a robot evaluate their consequences before acting. Learned world models provide this capability by modeling the dynamics of an agent's surroundings from experience [1], [2]. Joint-Embedding Predictive Architectures (JEPA) learn by predicting target representations from contextual information, without reconstructing observations [3]. Action-conditioned world models built on this principle predict future representations from current observations and candidate actions [4], [5], encouraging representations that capture how the environment evolves under actions. Such representations encode a form of *physical intuition*: the ability to anticipate the consequences of different actions.

To advance toward human-level intelligence, we must explore how models can use their understanding of action consequences to guide action selection. Theories of human motor control link prediction and action through paired internal models: one predicts how an action will change the body's state, while the other determines the action needed to produce a desired change [6]. This complementary ability to propose actions toward a goal is what we call *action intuition*. In JEPA-based world models, predictive representations are typically used to evaluate what an action will do; their potential to help generate actions remains less fully explored. This raises a question: how much action guidance can be extracted from these predictive representations?

To investigate this question, we learn a goal-conditioned action prior from the frozen encoder's current and goal embeddings, using a simple multilayer perceptron (MLP) as a lightweight implementation. This provides a direct way to test whether the existing features support goal-conditioned action prediction without adapting the world model. We then integrate the prior into sampling-based model predictive control (MPC) through closed-form Product-of-Gaussians (PoG) fusion [7]. The prior's mean and predicted uncertainty guide action sampling, while the world model evaluates candidate outcomes against the goal (Fig. 1). Together, these extraction and integration steps form **PRISM** (**PR**ior-guided **I**magination **S**ampling in world **M**odels), a framework for probing and exploiting action intuition in world-model representations. PRISM requires no additional visual encoder, pretrained policy, or language instructions, and supports different action-head architectures.

We assess PRISM at two levels across four simulated

control tasks—PushT, Cube, TwoRoom, and Reacher. First, simple MLP probes recover actions from consecutive observations encoded by the frozen world model, achieving R^2 values of 0.613–0.988 across the four tasks. Repeating the probes with a randomly initialized, frozen encoder and projector of the same architecture yields substantially lower scores, supporting the role of learned representations in action prediction. Second, planning-based evaluations test whether the extracted priors can improve control. Across three sampling budgets and ten evaluation seeds, PRISM improves mean success over the default prior-free MPC baseline on all four tasks by 11.8–35.4 percentage points. Comparisons on Franka PushT and ARX X5 cube manipulation also show higher success with PRISM. Together, these results highlight an underexplored capability of world-model representations: beyond predicting action consequences, they also support the action intuition.

II. Related Work

World models. World models predict the consequences of actions and support decision-making through imagined rollouts [1], [2]. DreamerV3 learns latent dynamics with an observation-reconstruction objective and trains an actor and value function using imagined experience [2]. Other approaches learn predictive representations without reconstructing observations. JEPA provides a framework for prediction in representation space [3]. DINO-WM and LeWM use latent predictions to evaluate action sequences against a visual goal [4], [5]. We build on LeWM, whose goal-conditioned planner scores trajectories using embedding distance without reward or value heads. Keeping its world model and scoring function fixed allows us to study how an action prior changes the planner's sampling distribution.

World models in sampling-based MPC. Sampling-based model predictive control (MPC) searches over action sequences by sampling candidates, predicting their outcomes, and updating the sampling distribution according to their scores. Model predictive path integral control (MPPI) [8] and the cross-entropy method (CEM) [9] are widely used approaches, with different weighting and distribution-update rules. TD-MPC2 combines a decoder-free latent world model with reward and value prediction for trajectory optimization [10], whereas DINO-WM and LeWM evaluate plans through visual goal distances [4], [5]. With a limited sampling budget, planning depends on both accurate predictions and useful action proposals. Biased-MPPI permits arbitrary sampling distributions and incorporates action sequences from ancillary controllers into its sample set [11]. PRISM focuses on extracting action priors from frozen world-model representations and evaluating their utility for planning.

Action priors for planner sampling. Learned action proposals can guide sampling in visual MPC. For example, VLMPC and its trajectory-generation extension use vision-language models to guide candidate action sampling [12], [13]. PiJEPA use a language-conditioned Octo policy to initialize planning [14]. Its policy generates actions from the current observation and language instruction, while the goal image enters the planning cost. PRISM instead learns a goal-conditioned action prior directly from current and goal embeddings, without a pretrained policy or language input. Our focus is the cost and utility of extracting such priors from existing world-model features, evaluated through action-prediction probes and controlled planning experiments.

III. Method

PRISM investigates whether representations learned to predict action consequences can also guide action selection. We first probe action predictability in a frozen world model, then learn an uncertainty-aware action prior from its features, and finally use this prior to guide planning (Fig. 2). The encoder and dynamics predictor remain fixed throughout.

A. Predictive representations and action readout

Frozen predictive representations. We build on an action-conditioned JEPA world model [5], comprising an encoder $h_\psi : \mathcal{O} \to \mathbb{R}^d$ and a latent dynamics predictor f_θ. The encoder maps current and goal observations to

$$z_t = h_\psi(o_t), \qquad z_g = h_\psi(o_g). \tag{1}$$

The predictor uses the current representation and an action sequence to predict the resulting future representation. This ability to anticipate action consequences constitutes physical intuition. Our question is whether the same representations also support action intuition: predicting actions that connect a current observation to a desired outcome.

Probing action information. Let a_t denote the action at environment step t and Δ the observation offset in environment steps. We test action predictability with a diagnostic MLP q_η that takes two frozen embeddings and predicts the first environment-step action:

$$\hat{a}_t = q_\eta([z_t, z_{t+\Delta}]), \qquad \mathcal{L}_{\text{probe}} = \mathbb{E}\left[\|\hat{a}_t - a_t\|_2^2\right]. \tag{2}$$

Only q_η is trained, using mean squared error (MSE). At $\Delta{=}1$, the probe predicts the action connecting consecutive observations, an inverse-dynamics readout. Larger offsets test whether the features support action prediction toward more distant observations. We train a probe for each task and offset $\Delta \in \{1, 5, 25\}$ and evaluate R^2 on held-out episodes (Sec. IV-A). These probes assess action predictability without providing planning priors.

B. Extracting an uncertainty-aware action prior

To use action intuition for control, we learn a goal-conditioned prior $p_\phi(\mathbf{a} \mid z_t, z_g)$ from the same task data used by the world model. The prior predicts an action-sequence distribution from current and goal embeddings, requiring no additional visual encoder or language instructions.

Lightweight Gaussian readout. Our main implementation uses an MLP g_ϕ to predict a diagonal Gaussian:

$$(\mu_p, \sigma_p) = g_\phi([z_t, z_g]), \qquad p_\phi = \mathcal{N}(\mu_p, \mathrm{diag}(\sigma_p^2)), \tag{3}$$

where $\mu_p, \sigma_p \in \mathbb{R}^{HBA}$ describe H planning steps, each containing B actions of dimension A. Unlike the diagnostic probes, this head learns an action distribution for planning.

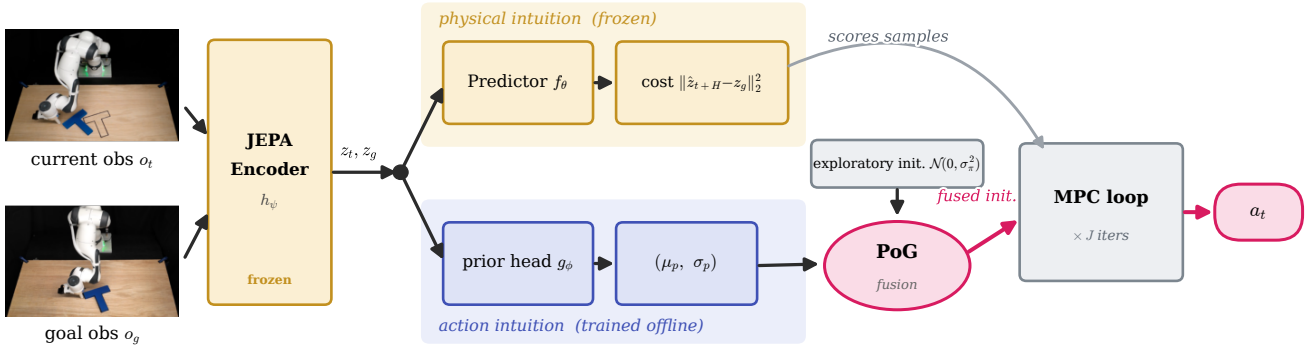


Fig. 2. **PRISM architecture.** The yellow group is outcome prediction, the blue group the action prior; both read the same frozen encoder, and only the prior head g_ϕ is trained. Grey marks the planner's default proposal, pink the fused distribution and the action it yields.

Head architecture and training. The MLP uses two 512-unit hidden layers with GELU activations and a $2HBA$-dimensional output for the mean and standard deviation, parameterized as $\sigma_p = \text{softplus}(r_\phi) + 0.05$, where r_ϕ is the raw output of the standard-deviation branch. Following Seitzer et al. [15], we train with the β-weighted negative log-likelihood (β-NLL) loss (β=0.5) on cached frozen features and normalized actions. The head has approximately 0.59M parameters on Cube and 0.51M on PushT; optimization settings are given in Sec. IV. PRISM also supports diffusion heads conditioned on the same frozen features; the mean and variance of their sampled action sequences are used for PoG fusion. All networks remain frozen during planning.

C. Using action intuition for world-model planning

Uncertainty-weighted fusion. At each replanning step, we combine the prior with the planner's default proposal $\mathcal{N}(\mu_\pi, \text{diag}(\sigma_\pi^2))$, where $\mu_\pi = 0$. Following the product-of-experts principle [7], multiplying the two diagonal Gaussians gives the per-coordinate update

$$\sigma^2_{\text{fused}} = (\tau_\pi + \tau_p)^{-1}, \tag{4}$$

$$\mu_{\text{fused}} = \sigma^2_{\text{fused}}(\tau_\pi \mu_\pi + \tau_p \mu_p), \tag{5}$$

where $\tau_\pi = \sigma_\pi^{-2}$ and $\tau_p = (s\sigma_p)^{-2}$ are elementwise precisions. The update depends only on $s\sigma_p/\sigma_\pi$; throughout we use the solver default σ_π=1.0 in normalized action space, s=1 for the MLP results, and floor σ_{fused} at 0.05.

The prior shifts the sampling mean and narrows the sampling distribution according to its predicted uncertainty. As $s\sigma_p$ increases, its influence diminishes and the fused proposal approaches the default distribution (Fig. 3).

Evaluating and refining actions. Model predictive control uses the fused proposal to sample candidate action sequences. The frozen world model predicts their outcomes and scores them by distance to the goal embedding:

$$\text{cost}(\mathbf{a}) = \|f_\theta(z_t, \mathbf{a}) - z_g\|_2^2. \tag{6}$$

Here $f_\theta(z_t, \mathbf{a})$ is the predicted terminal embedding. We use MPPI [8] to update the sampling mean by softmax-weighting the lowest-cost candidates, while keeping the fused covariance fixed within each replanning step. The controller then executes the first action and replans from the next observation. Thus, action intuition guides which candidates are explored, while physical intuition evaluates their predicted consequences. Algorithm 1 summarizes the planning procedure.

Algorithm 1 PRISM-MPPI planning step at environment time t

Require: frozen h_ψ, predictor f_θ, frozen head g_ϕ, prior scale s, horizon H, iterations J, samples K, elite count K_{elite}, softmax temperature $\lambda > 0$

$z_t \leftarrow h_\psi(o_t),\ \ z_g \leftarrow h_\psi(o_g)$

$(\mu_p, \sigma_p) \leftarrow g_\phi([z_t, z_g])$ ▷ action-intuition prior over HB environment-step actions

$(\mu_{\text{fused}}, \sigma_{\text{fused}}) \leftarrow \text{PoG}\big((\mu_\pi, \sigma_\pi), (\mu_p, s\sigma_p)\big)$ ▷ Eqs. 4–5

$\mu \leftarrow \mu_{\text{fused}}$ ▷ σ_{fused} held fixed for all iterations

for $j \leftarrow 1$ to J **do**

  Draw $\{\mathbf{a}^{(i)}\}_{i=1}^K \sim \mathcal{N}(\mu, \text{diag}(\sigma^2_{\text{fused}}))$

  $\text{cost}^{(i)} \leftarrow \|f_\theta(z_t, \mathbf{a}^{(i)}) - z_g\|_2^2$

  Retain the K_{elite} lowest-cost candidates

  $\mu \leftarrow \sum_i w_i\, \mathbf{a}^{(i)}, \quad w_i \propto \exp(-\text{cost}^{(i)}/\lambda)$ ▷ normalize weights over retained candidates

end for

return first action of μ

IV. Experiments

We evaluate action intuition at two levels: predicting demonstrated actions from frozen features (Sec. IV-A) and using an extracted action prior to improve planning (Sec. IV-B). Both evaluations cover four goal-conditioned visual control tasks: PushT, Cube, TwoRoom, and Reacher. Additional analyses on PushT and Cube examine the effects of prior design, training-data corruption, and planner choice.

Environments and datasets. PushT requires a planar pusher to align a T-block with a target pose (A=2) [16]; Cube is the `cube-single` robotic arm task from OGBench (A=5) [17]; TwoRoom is 2D navigation through a door between two rooms (A=2); Reacher is a two-joint arm reaching a target configuration (A=2). The four datasets

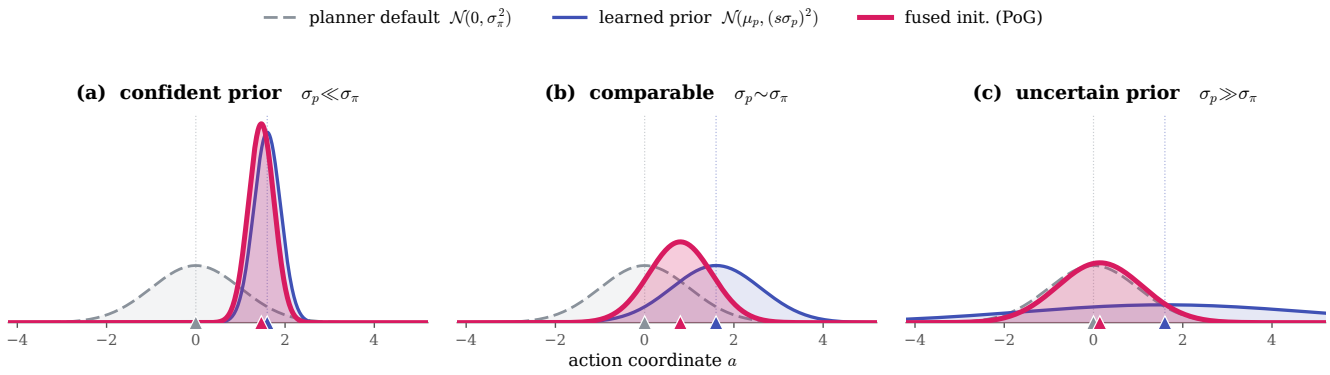


Fig. 3. **Uncertainty-weighted action sampling.** Grey is the default proposal, blue the action prior, pink their product; the panels vary the prior's precision relative to the planner's. Drawn before the standard-deviation floor is applied.

differ in demonstrator quality, which matters for how much action information survives to the planner's goal distance: PushT provides 18,685 Diffusion-Policy episodes (99.5% trajectory success), Cube 10,000 scripted-oracle episodes (100%), TwoRoom 10,000 episodes from a noisy two-stage heuristic (40.4%), and Reacher 10,000 episodes in which no episode reaches the success criterion. On each task the world model and the action head use the same dataset and the same frozen LeWM encoder [5]. The deployed head is trained with β-NLL as described in Sec. III-B; the diagnostic probe protocol is specified below.

Head optimization. The deployed heads use AdamW (learning rate 3×10^{-4}, weight decay 10^{-4}), batch size 256, and 50 epochs, with a 1,000-step linear warm-up followed by cosine decay. We split episodes $90/10$ for training and validation and select the lowest-validation-NLL checkpoint. Training windows contain HB valid actions and exclude invalid terminal actions. The same StandardScaler action normalization is used during training and deployment.

Simulation implementation. The deployed-head episode splits are $16{,}817/1{,}868$ for PushT and $9{,}000/1{,}000$ for Cube, TwoRoom, and Reacher. Simulation planning uses J=30 iterations and softmax temperature λ=0.5, with the task-specific success criteria used by LeWM. All simulation experiments run on a single RTX 5090 with CUDA 12.8. Figure 4 shows one observation–goal pair per task.

A. Action prediction from frozen representations

We fit a separate MLP probe to predict a_t from $[z_t, z_{t+\Delta}]$ for each task and temporal offset $\Delta \in \{1, 5, 25\}$. At Δ=1, this is an inverse-dynamics prediction; Δ=25 matches the temporal offset used for planning goals. Each probe uses a two-layer MLP with hidden width 512 and GELU activations, trained with MSE on StandardScaler-normalized actions. Episodes are split $90/10$ by index, with 250,000 training windows and 40,000 held-out windows sampled per fit. We use AdamW (learning rate 3×10^{-4}, weight decay 10^{-4}), batch size 512, and 12 epochs. We select the epoch with the lowest validation MSE and report its validation coefficient of determination, $R^2 = 1 - \mathrm{MSE}/\mathrm{Var}(a_t)$, averaged over three probe seeds $\{0, 1, 42\}$. Because actions are StandardScaler-normalized, $\mathrm{Var}(a_t) \approx 1$, and R^2=0 is the score of a predictor that always emits the dataset-mean action. These are validation scores on held-out episodes, not scores from an additional independent test split. Note that R^2 scores the conditional mean and is reduced by magnitude shrinkage: a prediction pointing in the correct direction at a fraction of the true magnitude scores low while still displacing a sampling distribution.

TABLE I

ACTION PREDICTION FROM FROZEN FEATURES. PROBE R^2 (MEAN±STD OVER THREE SEEDS; 40,000 HELD-OUT WINDOWS PER FIT). THE LOWER BLOCK USES A RANDOMLY INITIALIZED, FROZEN ENCODER AND PROJECTOR WITH THE SAME ARCHITECTURE AND PROBE PROTOCOL. $R^2 = 0$ DENOTES MEAN-ACTION PREDICTION.

Task	Δ=1	Δ=5	Δ=25
Trained LeWM encoder (frozen)			
PushT	$.798 \pm .009$	$.751 \pm .003$	$.367 \pm .010$
Cube	$.739 \pm .001$	$.699 \pm .001$	$.628 \pm .001$
TwoRoom	$.988 \pm .000$	$.219 \pm .003$	$.087 \pm .002$
Reacher	$.613 \pm .001$	$.215 \pm .001$	$.031 \pm .001$
Same architecture, re-initialized from scratch			
PushT	$.000 \pm .000$	$-.000 \pm .000$	$.000 \pm .001$
Cube	$.090 \pm .002$	$.168 \pm .002$	$.094 \pm .001$
TwoRoom	$.000 \pm .000$	$.000 \pm .000$	$.002 \pm .000$
Reacher	$.000 \pm .000$	$.000 \pm .000$	$.000 \pm .000$

Table I shows that a simple readout can recover actions from one-step transitions in frozen feature space on all four tasks (R^2=0.613–0.988). Predictability decreases with temporal distance: Cube and PushT retain stronger signals at Δ=25, whereas TwoRoom and Reacher have much weaker long-offset predictions. The lower block of Table I repeats every fit with a randomly initialized encoder of the same architecture: nine of the twelve cells fall to R^2=0, and the one task that keeps a residual shows no decay in Δ. What the probes read is therefore a property of the predictive training, not of any 192-dimensional visual embedding.

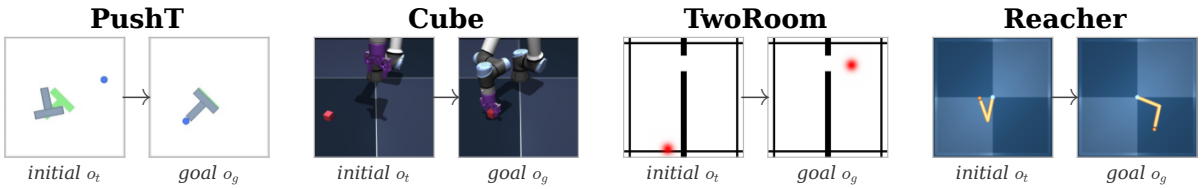


Fig. 4. **Simulation tasks.** One episode's first and last observation per task; at evaluation the goal is instead drawn 25 environment steps ahead of the start. In PushT the grey T-block must reach the green pose and the blue disk is the pusher; TwoRoom renders only the agent, so its goal image is the agent at the target.

TABLE II

EXECUTING THE PRIOR WITHOUT THE PLANNER. SR (%, MEAN±STD) OVER TEN SEEDS. NO CANDIDATES ARE SAMPLED, SO THERE IS NO K; THE PRISM COLUMN IS REPEATED FROM FIG. 5 FOR REFERENCE.

Task	Mean-only execution	PRISM-MPPI (K=32)
PushT	28.2 ± 5.9	80.6 ± 5.6
Cube	68.0 ± 5.6	82.8 ± 5.5
TwoRoom	75.6 ± 4.4	81.8 ± 5.8
Reacher	7.0 ± 4.3	60.0 ± 6.9

TABLE III

COST AND PERFORMANCE OF ACTION PRIORS. SR (%, MEAN±STD) AT K=32 OVER TEN SEEDS, 50 EPISODES EACH. BOTH LEARNED PRIORS USE THE SAME WORLD MODEL, PLANNER AND POG RULE, BUT DIFFERENT TRAINING-GOAL CONVENTIONS (SEE TEXT). TRAINING TIMES ON ONE RTX 5090 EXCLUDE WORLD-MODEL PRETRAINING AND FEATURE CACHING; INFERENCE IS PRIOR GENERATION PER REPLAN, EXCLUDING THE SHARED ENCODER AND PLANNER.

Prior	Par. (M)	Train (min)	Infer (ms)	PushT	Cube
None	0	–	–	56.0 ± 5.7	47.4 ± 6.3
MLP head	0.51–0.59	0.4–3.1	0.048	80.6 ± 5.6	82.8 ± 5.5
Diffusion head	18.5	23	90–92	93.6 ± 3.6	99.8 ± 0.6

B. Planning with the extracted action prior

Evaluation protocol. We compare three planners on four tasks at $K \in \{32, 64, 128\}$: *vanilla MPPI*, which initializes at zero mean and the default width σ_π; *warm-start MPPI*, which initializes the mean at μ_p but keeps σ_π and so discards the prior's σ_p; and *PRISM-MPPI*, which uses both through the PoG update of Eqs. (4)–(5). Each configuration uses 50 episodes per evaluation seed; we report mean±std across ten seeds $\{0, \ldots, 9\}$. All tasks use a goal offset of 25 environment steps, a planning horizon of H=5, and action blocks of B=5. K is the number of candidate action sequences per iteration; all three planners use the same elite rule, K_{elite}=30, and the prior scale is s=1 for PRISM. Warm-start discards σ_p, so s does not enter that baseline. Rerunning both baselines at $K_{\text{elite}}=K$ moves every cell by at most 1.4 pp (24 cells, ten paired seeds; pooled +0.09 pp, p=0.63), so no comparison below turns on the elite count.

Fig. 5 shows higher mean success for PRISM than for the default prior-free baseline in all twelve task–budget settings, with gains of 11.8–35.4 percentage points. PRISM also exceeds mean-only warm-starting in every setting. At K=32, PRISM outperforms vanilla MPPI at K=128 on all four tasks — paired over the same ten seeds, +21.8 pp on PushT, +34.6 pp on Cube, +8.4 pp on TwoRoom and +16.2 pp on Reacher, winning on 10/10 seeds in every case ($p \leq 1.1 \times 10^{-3}$) — so the gain comes from where sampling starts rather than from how many samples are drawn. Executing the prior's mean without candidate sampling achieves 7.0–75.6% success across tasks and underperforms PRISM on all four (Table II), showing the benefit of using the prior within a planner.

C. Cost and performance of action priors

We compare the MLP with an 18.5M-parameter conditional UNet1D diffusion head based on Diffusion Policy [16], including an 18.3M-parameter denoiser. It predicts HB=25 actions from the same frozen embeddings $[z_t; z_g]$. We estimate the prior mean and standard deviation from 16 denoising diffusion implicit model (DDIM) [18] samples, each using 10 denoising steps, and apply the same PoG update and planner. The diffusion head uses goals 25 steps ahead, while the MLP uses terminal frames; this comparison therefore does not isolate head architecture.

We also compare frozen-feature and pixel-conditioned diffusion training, holding the training windows, goal convention, schedule, and step count fixed. The pixel-conditioned policy has 19.3M parameters and trains its own CNN. On PushT and Cube, respectively, validation denoising losses are 0.0187 and 0.0774 for frozen features, versus 0.0206 and 0.0767 for pixels. Frozen-feature head training takes approximately 23 minutes per task, versus 254 and 415 minutes for the pixel-conditioned policy. The 23-minute figure excludes feature extraction, which takes 574 s on PushT and 573 s on Cube and is performed once for the cached features shared by the MLP and diffusion heads. Thus, frozen features yield comparable validation denoising losses while reducing diffusion-head training time; this comparison excludes world-model pretraining.

Both priors improve mean success over prior-free planning (Table III). The diffusion head achieves higher success, while the MLP uses 31–36× fewer parameters and substantially less training and inference time.

End-to-end planning latency on Cube at K=128 (RTX 5090) is 210.1 ± 1.2 ms for vanilla MPPI, $210.7 \pm$

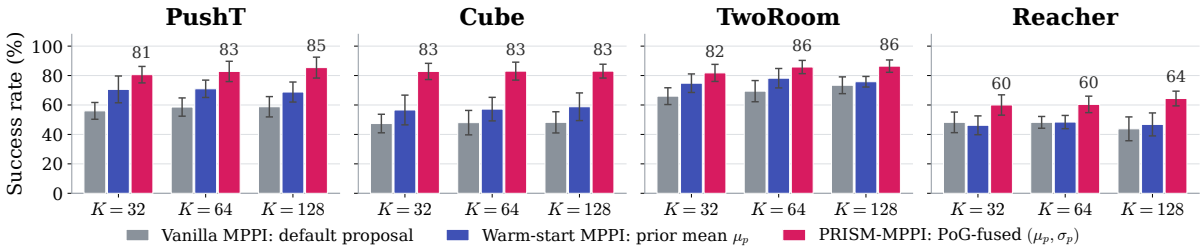


Fig. 5. **Four-task planning results.** Bars are means over ten evaluation seeds, whiskers one standard deviation; PRISM-MPPI means are printed above their bars. Colours carry the same meaning as in Fig. 3: grey the default proposal, blue the learned prior, pink the PoG-fused distribution.

TABLE IV

TWO WAYS TO INJECT ONE PRIOR. SR (%, MEAN±STD) AT K=32 OVER TEN SEEDS, BOTH RULES FED THE SAME MLP HEAD. *Weak* RETRAINS THE HEAD ON ε=50% CORRUPTED DEMONSTRATIONS. BOLD MARKS THE BETTER RULE; CUBE'S WEAK CELL IS A TIE IN THE MEAN.

Prior	Rule	PushT	Cube
Clean	PiJEPA-style	77.0 ± 4.5	81.2 ± 6.4
Clean	PoG	$\mathbf{80.6 \pm 5.6}$	$\mathbf{82.8 \pm 5.5}$
Weak	PiJEPA-style	45.0 ± 7.2	51.8 ± 3.9
Weak	PoG	$\mathbf{62.0 \pm 5.4}$	51.8 ± 6.0

2.2 ms for mean-only initialization, and 211.6 ± 4.8 ms for PRISM. The measured overhead is within the run-to-run variation of the shared planning loop.

D. PoG against a PiJEPA-style initialization

PiJEPA initializes the planner from the empirical mean and clamped standard deviation of its policy's samples [14], i.e. it *replaces* the default proposal rather than multiplying with it. This provides a direct alternative to PoG fusion, which we evaluate using the same MLP prior. PiJEPA clamps the standard deviation to $[\sigma_{\min}, \sigma_{\max}]$ and reports $[0.01, 0.05]$ in its own action normalization, where its prior-free baseline is initialized at $\sigma^0{=}\sigma_{\max}$; the upper clamp is therefore that planner's default sampling width, and we set $\sigma_{\max} = \sigma_\pi$ accordingly. We evaluate with the deployed head and with the head retrained on 50% corrupted demonstrations (Sec. IV-F).

PoG is ahead in three of the four cells (Table IV), and the one substantial gap is the weak prior on PushT, +17.0 pp on matched seeds and 10/10 seed wins. Cube's weak cell is a tie. This difference is consistent with the larger increase in predicted uncertainty on PushT: corrupting half the demonstrations raises σ_p from 0.53 to 3.08 on PushT but only from 0.60 to 1.18 on Cube (Table VII). As predicted uncertainty increases, PoG reduces the prior's weight in the fused sampling mean. The replace rule has no such term: it installs μ_p at full strength whatever σ_p reports. Running the same comparison with a diffusion prior instead of the head gives +0.4 to +5.0 pp, within seed noise.

Encoder ablation. Inspired by DINO-WM [4], we evaluate our own DINOv2-based world-model implementation. Swapping the ViT-tiny encoder for frozen DINOv2-base with a learnable $768 \to 192$ projector, holding the LeWM predictor and training recipe fixed, PRISM keeps a +6 to +18 pp lift over vanilla MPPI (5/6 cells, one-sided paired t; Table V, three seeds on PushT and Cube). The comparison is within-encoder: absolute levels differ between the two encoders, and the fusion does not depend on which frozen representation it reads.

E. The prior transfers to CEM

We evaluate whether the extracted action prior also improves CEM. Using the same prior head, we compare prior-free sampling, mean-only initialization, and PoG initialization on PushT and Cube. Unlike MPPI, CEM updates both the sampling mean and covariance during optimization.

PoG initialization improves mean success over prior-free CEM at all three sampling budgets on both tasks (Table VI). On Cube, it also outperforms mean-only initialization at every budget. On PushT, however, mean-only initialization performs better at $K = 32$ and 64, while the two achieve the same mean success at $K = 128$. These results show that the extracted prior can benefit CEM, although PoG does not consistently outperform mean-only initialization. On PushT at $K = 32$, sweeping the initial standard-deviation floor over $\{0.05, 0.10, 0.20, 1.0\}$ yields 24.0–25.3% success. Increasing the initial floor therefore does not remove the gap to mean-only initialization; it does not constrain CEM's subsequent covariance refits.

F. Increased uncertainty reduces reliance on corrupted priors

We test how the prior responds to corrupted training data. For a fraction ε of training episodes, we replace the action labels with uniform random samples within the action bounds and retrain the head. The world model and planning configuration remain fixed.

As corruption increases, both the head's mean predicted standard deviation and its held-out prediction error increase (Table VII). The predicted standard deviation rises from 0.53 to 4.49 on PushT and from 0.60 to 1.51 on Cube, reducing the prior's precision and its influence on sampling. With all action labels corrupted, planning success is 60.7% on PushT and 46.7% on Cube, close to the respective prior-free results of 58.8% and 48.2%. These comparisons use different seed sets, so the agreement is approximate. The results show that, under action-label corruption, the head predicts greater

TABLE V

ENCODER ABLATION. SR (%, MEAN±STD) OVER THREE SEEDS $\{0, 1, 42\}$ ON PUSHT AND CUBE. *Learned-prior params* COUNTS THE TRAINABLE ACTION PRIOR ONLY; EVERY ROW ALSO USES THE SHARED FROZEN WORLD MODEL.

Method	Learned-prior params	PushT			Cube		
Planning with World Model		K=32	K=64	K=128	K=32	K=64	K=128
Vanilla MPPI (DINOv2-based)	0	4 ± 2	3 ± 2	5 ± 1	45 ± 4	49 ± 6	41 ± 3
PRISM-MPPI (DINOv2-based)	0.51–0.59M	13 ± 6	10 ± 7	15 ± 5	57 ± 7	65 ± 5	67 ± 3
Vanilla MPPI (LeWM) [5]	0	59 ± 5	61 ± 7	57 ± 6	46 ± 2	44 ± 5	44 ± 4
PRISM-MPPI (ours)	0.51–0.59M	$\mathbf{82 \pm 4}$	$\mathbf{86 \pm 6}$	$\mathbf{89 \pm 4}$	$\mathbf{79 \pm 2}$	$\mathbf{78 \pm 3}$	$\mathbf{79 \pm 6}$

TABLE VI

THE SAME PRIOR UNDER CEM. SR (%, MEAN±STD) OVER THREE SEEDS $\{0, 1, 42\}$, 50 EPISODES PER SEED, ON PUSHT AND CUBE. ALL THREE ROWS USE CEM, WHICH REFITS σ EACH ITERATION; THE MPPI COUNTERPARTS ARE IN FIG. 5, MEASURED OVER TEN SEEDS AND THEREFORE NOT DIRECTLY COMPARABLE CELL BY CELL.

Task	K	No prior	Mean only	PoG
PushT	32	12.7 ± 8.1	$\mathbf{41.3 \pm 3.1}$	25.3 ± 3.1
PushT	64	80.7 ± 8.1	$\mathbf{88.7 \pm 6.1}$	84.0 ± 7.2
PushT	128	90.7 ± 6.1	92.7 ± 5.8	$\mathbf{92.7 \pm 2.3}$
Cube	32	53.3 ± 6.4	60.7 ± 1.2	$\mathbf{68.7 \pm 7.0}$
Cube	64	59.3 ± 8.3	74.7 ± 2.3	$\mathbf{79.3 \pm 6.4}$
Cube	128	61.3 ± 4.6	76.0 ± 4.0	$\mathbf{82.7 \pm 5.0}$

TABLE VII

DEGRADED DEMONSTRATIONS. σ_p IS THE HEAD'S MEAN PREDICTED STANDARD DEVIATION ON HELD-OUT DATA; VAL MSE IS ITS HELD-OUT ERROR. SR AT K=128 OVER THREE SEEDS, EXCEPT THE ε=0 AND *no prior* ROWS, WHICH ARE THE TEN-SEED VALUES OF FIG. 5.

	PushT			Cube		
ε	σ_p	val MSE	SR	σ_p	val MSE	SR
0%	0.53	0.73	85.4	0.60	0.61	83.0
25%	1.96	0.79	72.0	0.96	0.68	52.7
50%	3.08	1.07	66.7	1.18	0.77	52.7
75%	3.90	1.56	66.0	1.33	0.90	50.7
100%	4.49	2.16	60.7	1.51	1.04	46.7
no prior	—	—	58.8	—	—	48.2

uncertainty and PoG correspondingly reduces reliance on the prior.

G. Sensitivity to the prior scale

We vary the prior scale s on PushT and Cube at $K = 128$, using three seeds $\{0, 1, 42\}$ (Fig. 6). Performance peaks around $s = 0.3$ to 1, supporting our default choice of $s = 1$. Larger values reduce the prior's influence, and at $s = 10^4$, success rates approach the prior-free results: 43% versus 44% on Cube and 59% versus 57% on PushT.

This trend is consistent with the PoG proposal approaching the default distribution as s increases; in the limit the two planners are the same algorithm, since they now share every other setting.

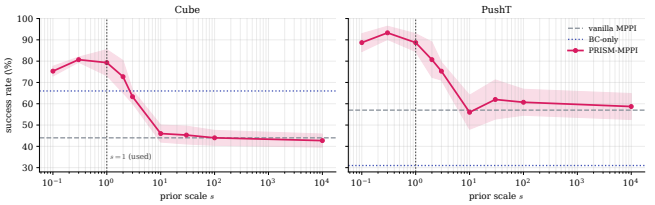


Fig. 6. **Prior-scale sweep.** SR against the prior scale s (log axis), mean±std over three seeds. The dashed and dotted lines are vanilla MPPI and mean-only execution on the same task.

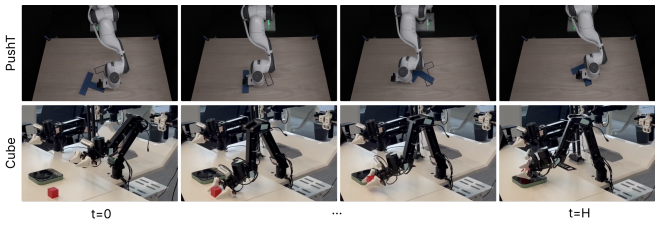


Fig. 7. **Real-robot rollouts.** Keyframes (left→right) of one successful PRISM-MPPI episode per platform. *Top:* Franka PushT. *Bottom:* ARX X5 cube manipulation.

H. Real-Robot Evaluation

We evaluate PRISM on two real-robot manipulation tasks: T-block pushing with a Franka Research 3 (FR3) arm and cube manipulation with an ARX X5 arm (Fig. 7). Each platform uses a third-person RGB camera and a world model trained on task-specific teleoperation data. We compare planning with and without the learned prior over 40 trials per condition, interleaved in randomized order, using $s = 1$.

PRISM achieves higher observed success on both platforms: $14/40$ versus $10/40$ on Franka PushT, and $18/40$ versus $12/40$ on ARX cube manipulation. These results are consistent with the improvements observed in simulation, although more trials are needed to establish the size and reliability of the gains.

Hardware and data. The FR3 setup uses a RealSense D455, Meta Quest 3 teleoperation, and an RTX 5080 for planning. The ARX X5 setup uses an Agilex Mobile ALOHA platform, a RealSense D435i, joystick teleoperation, and an RTX 4090. Neither setup uses wrist or egocentric cameras (Fig. 8). The datasets contain 411 episodes for Franka PushT and 201 for ARX cube manipulation.

Franka data and control protocol. The FR3 uses Cartesian-

impedance control with planar end-effector displacements $(\Delta x, \Delta y)$; height, wrist orientation, and gripper are held fixed. RGB images are resized to 224×224. The Franka dataset contains 93,728 frames recorded at 10 Hz, comprising 200 target-completion episodes and 211 episodes ending at arbitrary positions. The world model uses all 411 episodes, while the action head uses only the 200 target-completion episodes with their terminal images as goals.

For this setup, planning uses K=300, J=30, H=3, and B=5, giving 1.5 s of lookahead. Each replan starts from default scale $\sigma_\pi = 1.0$, fuses the prior, and retains the resulting covariance during optimization. A goal image is captured once per session. The controller executes five denormalized actions at 10 Hz before replanning, giving a 0.5 s replanning interval. These control settings apply to the Franka setup.



(a) Franka FR3 + PushT (b) ARX X5 + cube

Fig. 8. **Real-robot setups.** The red circle marks the tripod-mounted RealSense that is the planner's only RGB sensor; neither platform uses a wrist or egocentric camera. *(a)* FR3, D455: the blue T-block must reach the marked target outline. *(b)* ARX X5 on an Agilex Mobile ALOHA, D435i: the arm transports a single red cube.

V. Limitations

Action predictability weakens for distant goals, and inaccurate, overconfident priors can impair planning. The utility of uncertainty-guided sampling depends on the task and the quality of the learned prior. PoG may discard multimodal structure, and future work will explore fusion methods that better preserve prior structure and adapt to prior reliability. Real-robot results use 40 trials per condition and require broader validation.

VI. Conclusion

We showed that representations learned to predict action consequences also support action intuition: goal-conditioned action priors can be learned from frozen world-model features through a lightweight readout. Action-prediction probes across four tasks, together with same-architecture random-initialization controls, provide evidence that predictive training yields features useful for action prediction. Through PRISM, we further demonstrated that the extracted priors improve planning performance in simulated and real-robot tasks without retraining the world model. Together, these findings reveal an underexplored capability of predictive representations: the same features can support both anticipating action consequences and proposing goal-directed actions.

Acknowledgments

OpenAI's ChatGPT/Codex tools were used for drafting, restructuring, and language editing throughout, from author-provided descriptions and experimental results. The authors are responsible for the final content.

References

[1] D. Ha and J. Schmidhuber, "World models," *arXiv:1803.10122*, 2018.

[2] D. Hafner, J. Pasukonis, J. Ba, and T. Lillicrap, "Mastering diverse control tasks through world models," *Nature*, vol. 640, pp. 647–653, 2025.

[3] Y. LeCun, "A path towards autonomous machine intelligence," OpenReview, 2022, position paper, version 0.9.2.

[4] G. Zhou, H. Pan, Y. LeCun, and L. Pinto, "DINO-WM: World models on pre-trained visual features enable zero-shot planning," in *Int. Conf. Mach. Learn. (ICML)*, 2025.

[5] L. Maes, Q. Le Lidec, D. Scieur, Y. LeCun, and R. Balestriero, "LeWorldModel: Stable end-to-end joint-embedding predictive architecture from pixels," *arXiv:2603.19312*, 2026.

[6] D. M. Wolpert and M. Kawato, "Multiple paired forward and inverse models for motor control," *Neural Netw.*, vol. 11, no. 7–8, pp. 1317–1329, 1998.

[7] G. E. Hinton, "Training products of experts by minimizing contrastive divergence," *Neural Comput.*, vol. 14, no. 8, pp. 1771–1800, 2002.

[8] G. Williams, N. Wagener, B. Goldfain, P. Drews, J. M. Rehg, B. Boots, and E. A. Theodorou, "Information theoretic MPC for model-based reinforcement learning," in *IEEE Int. Conf. Robot. Autom. (ICRA)*, 2017.

[9] P.-T. De Boer, D. P. Kroese, S. Mannor, and R. Y. Rubinstein, "A tutorial on the cross-entropy method," *Ann. Oper. Res.*, vol. 134, no. 1, pp. 19–67, 2005.

[10] N. Hansen, H. Su, and X. Wang, "TD-MPC2: Scalable, robust world models for continuous control," in *Int. Conf. Learn. Represent. (ICLR)*, 2024.

[11] E. Trevisan and J. Alonso-Mora, "Biased-MPPI: Informing sampling-based model predictive control by fusing ancillary controllers," *IEEE Robot. Autom. Lett.*, vol. 9, no. 6, pp. 5871–5878, 2024.

[12] W. Zhao, J. Chen, Z. Meng, D. Mao, R. Song, and W. Zhang, "VLMPC: Vision-language model predictive control for robotic manipulation," in *Robot.: Sci. Syst. (RSS)*, 2024.

[13] J. Chen, W. Zhao, Z. Meng, D. Mao, R. Song, W. Pan, and W. Zhang, "Vision-language model predictive control for manipulation planning and trajectory generation," *arXiv:2504.05225*, 2025.

[14] A. Chahe and L. Zhou, "Policy-guided world model planning for language-conditioned visual navigation," *arXiv:2603.25981*, 2026.

[15] M. Seitzer, A. Tavakoli, D. Antic, and G. Martius, "On the pitfalls of heteroscedastic uncertainty estimation with probabilistic neural networks," in *Int. Conf. Learn. Represent. (ICLR)*, 2022.

[16] C. Chi, S. Feng, Y. Du, Z. Xu, E. Cousineau, B. Burchfiel, and S. Song, "Diffusion policy: Visuomotor policy learning via action diffusion," in *Robot.: Sci. Syst. (RSS)*, 2023.

[17] S. Park, K. Frans, B. Eysenbach, and S. Levine, "OGBench: Benchmarking offline goal-conditioned RL," in *Int. Conf. Learn. Represent. (ICLR)*, 2025.

[18] J. Song, C. Meng, and S. Ermon, "Denoising diffusion implicit models," in *Int. Conf. Learn. Represent. (ICLR)*, 2021.